AirTalk

Gesture-Powered Assistive Communication

Project Report & Technical Documentation

Field	Details
Developer	Affan Sajid
Collaborator	Sidharth
Event	Google Physical AI Hackathon 2026
Status	Version 1.0 — Working Prototype
Repository	github.com/Affan-sajid/airtalk
Platform	macOS (Apple Silicon), Arduino-compatible hardware
Date	April 20, 2026

1. Abstract

AirTalk is an assistive communication device that lets a person write words in the air using a pen-shaped wand, then translates those gestures into natural spoken sentences in real time. The system combines a high-precision inertial measurement unit (IMU), a CNN-based word classifier, a large language model for sentence reconstruction, and text-to-speech output — all in a pipeline that runs end-to-end on commodity hardware.

This report covers the motivation, hardware design, software architecture, machine learning methodology, and the development journey from first prototype to hackathon demonstration. It also sets out the roadmap for Version 2, where the word-by-word classifier gives way to a direct Teeline shorthand translation model — a fundamentally more powerful approach that was conceived during the hackathon but requires dedicated time to build properly.

2. The Problem

Over 430 million people worldwide are deaf, hard of hearing, or speech-impaired. For many, sign language is the primary mode of communication — but sign language is not universally understood. In most everyday interactions, a non-signing person is present, and communication breaks down immediately.

The gap

The problem is not that sign language is hard to learn. The problem is that existing communication systems were designed for humans to interpret — not for machines to recognise. That design choice limits scalability at a fundamental level.

Existing technology-based solutions have not closed this gap. Camera-based sign language translators require full body tracking, consistent lighting, and fixed camera positions. Haptic glove systems are bulky, expensive, and socially conspicuous. Neither fits naturally into daily life.

The deeper issue is that both approaches attempt to translate a system built for human perception (sign language) into a form machines can understand. AirTalk takes the opposite approach: design a gesture system that is optimised for machines from the ground up, then make it intuitive enough for humans to learn quickly.

3. The Solution

3.1 Core Concept

AirTalk is a wand-shaped device, roughly the size and weight of a thick pen, that a user holds in one hand. They draw gesture shapes in the air — letters, abbreviated words, or shorthand symbols — and the device captures that motion, classifies it, and converts it into speech.

The result is a communication interface that requires only one hand, one finger's worth of movement, no camera, no trained interpreter, and no body tracking. It works in the dark, in a hospital, in a crowded room, or in complete silence.

3.2 Inspiration: Teeline Shorthand

The gesture vocabulary is inspired by Teeline shorthand — a fast writing system developed for journalists that strips letters down to their most essential strokes. Letters combine into words efficiently: H and I traced together become HI; adding a word in front produces a full phrase.

Adapting this into air gestures means the system is learnable in a structured way. There is a logic to each gesture. New vocabulary is additive and consistent. In Version 1, the classifier recognises a set of core words. The longer-term vision is a model that reads

Teeline shorthand strokes directly and continuously — translating fluid writing rather than discrete word-by-word inputs.

3.3 End-to-End Pipeline

The full data flow from gesture to speech is:

- User holds the wand and draws a word gesture in the air while pressing the button
- The IMU samples motion at 100 Hz, streaming gyroscope and accelerometer data over USB
- Python renders the motion as strokes on a virtual canvas in real time
- On button release, the canvas is captured as an image and fed to the classifier
- The CNN returns a word label and confidence score
- Accumulated words are collected across multiple strokes
- After a brief idle period, the word list is sent to a language model for sentence reconstruction
- The reconstructed sentence is spoken aloud via text-to-speech

4. Hardware Architecture

4.1 Components

Component	Specification	Role
Microcontroller	ESP32 / Seeed XIAO (Arduino-compatible)	Sensor polling, button state, serial/USB transmission
IMU	Adafruit ISM330DHCX (STMicroelectronics)	6-DoF: 3-axis gyroscope + 3-axis accelerometer at 100 Hz
Button	Tactile switch on GPIO pin 1 (INPUT_PULLUP)	Pen-down / pen-up signal; calibration trigger
Communication	USB serial at 115,200 baud	Streams one CSV row per 10 ms loop cycle to host machine

The ISM330DHCX was chosen over consumer-grade alternatives such as the MPU-6050 because it is rated for high-vibration industrial environments. In practice this means the sensor produces significantly less baseline jitter, which matters when the user is making the fine wrist movements involved in drawing letter shapes.

4.2 Data Format

The firmware sends one comma-separated row every 10 ms containing six values: rotX, rotY, linX, linY, penDown, and accelBiasZ. All motion values are scaled by a factor of 100 before transmission to avoid floating-point formatting overhead, and divided by 100 in Python to restore physical units.

4.3 Calibration

Calibration happens on the falling edge of the button press — the first moment the user presses down. One IMU snapshot is taken at that instant and stored as the zero-bias for all six sensor channels. Every subsequent reading subtracts this bias. This means calibration is automatic, instant, and aligned to the user's natural grip position at the start of each stroke — no startup ritual, no holding still for two seconds.

5. Software Architecture

5.1 Motion Processing

Raw sensor data is processed through a two-component motion model. Gyroscope readings provide the primary cursor movement — they are clean, low-latency, and capture the fine wrist rotations that form the shapes of letters. Accelerometer readings provide translational movement — the ability to slide the whole hand across the canvas rather than just rotating the wrist.

A gravity leakage estimator runs continuously to separate intentional hand translation from the constant gravitational acceleration. A rotation gate prevents phantom velocity from accumulating when the pen is rotating rapidly. Velocity damping bleeds off drift when the hand is still. The result is a cursor that responds predictably to deliberate movements and ignores the small tremors and vibrations of a real hand.

5.2 Canvas and Stroke Capture

Each button press opens a fresh 900 x 650 pixel canvas and recentres the cursor. While the button is held, white strokes are drawn against a dark background in real time. On release, the canvas is captured as a PNG and passed to the classifier. The next press clears the canvas and starts again.

This image-based approach was a deliberate design choice. By rendering strokes as images, the classifier can use any standard image classification architecture without modification. The bounding box crop at preprocessing time compensates for the fact that different users draw at different scales and positions on the canvas.

5.3 Inference Pipeline

The classifier loads lazily on first use. On pen release, the canvas PNG is preprocessed: a tight bounding box is computed around the non-background pixels, the crop is padded slightly, resized to 64 x 64 pixels, normalised, and passed to the model. The prediction label and confidence percentage are printed to the terminal and added to the session word list.

5.4 Language Model and Speech

After a configurable idle period following the last recognised word, the accumulated word list is sent to a language model with a structured prompt that asks for a single, grammatically correct sentence. The prompt includes few-shot examples so the model understands the task: the input "hi name affan" should produce "Hi, my name is Affan." — not a paraphrase, not an explanation, just the natural sentence.

For known high-frequency phrases, a shortcut dictionary bypasses the language model entirely and returns a hardcoded sentence. This eliminates latency and API dependency for the most common utterances. The phrase matching uses unordered set comparison so the words can be drawn in any sequence.

The reconstructed sentence is spoken aloud using the macOS text-to-speech engine and displayed as an overlay on the canvas for eight seconds.

6. Machine Learning

6.1 Model: MobileNetV2

The classifier uses MobileNetV2, a convolutional neural network architecture designed for efficient on-device inference. Its depthwise separable convolutions reduce parameter count from the ~25 million of a standard ResNet-50 to approximately 3.4 million, while maintaining competitive accuracy on image classification tasks. The classification head was replaced with a linear layer sized to the number of word classes.

6.2 Version 1 Scope

The current classifier recognises ten word classes. These were chosen to enable a small but meaningful demonstration vocabulary: I, affan, and, communicate, mouth, name, not, sidharth, thank you, we. The training dataset totals approximately 421 images captured using the device itself, organised by class.

Class	Sample count
we	68
communicate	48
mouth	49

Class	Sample count
and	50
name	38
not	37
affan	33
thank you	36
I	31
sidharth	31

V1 note *The classifier is proof-of-concept scaffolding. Ten words are enough to demonstrate the pipeline. The long-term model is not a word classifier at all — see Section 8 on the roadmap.*

6.3 Preprocessing

Before classification, each captured canvas is processed as follows: the image is converted to greyscale, a tight bounding box is computed around all pixels brighter than a threshold value, the crop is padded by 12 pixels on each side, and the result is resized to 64 x 64 pixels using Lanczos resampling. The image is then normalised and expanded to three channels to match MobileNetV2's expected input format.

The bounding box crop is the most important step. It ensures the model sees the stroke shape at a consistent scale regardless of how large or small the user drew the gesture, and regardless of where on the canvas the stroke landed.

7. Development Journey

AirTalk was built during a hackathon sprint. The following is a summary of the major phases and the decisions made at each stage.

Phase	Key work
Hardware bring-up	IMU wiring, I2C address resolution, 100 Hz firmware, button pull-up fix

Phase	Key work
Visualisation	Real-time pygame canvas, cursor motion model, axis debugging and inversion fix
Calibration design	Moved from 2-second startup hold to instant per-press calibration
Serial reliability	Replaced six separate Serial.print() calls with single snprintf + flush to fix USB CDC fragmentation
Data collection	Built dedicated capture script with session labelling and auto-indexed filenames
Model training	MobileNetV2 training notebook, bounding box preprocessing, 80/20 train/val split
Inference integration	Lazy model loading, pen-release trigger, session word accumulation
LLM + TTS	Sentence reconstruction prompt, structured JSON output, phrase shortcuts, speech output
System hardening	LLM/TTS gate to prevent concurrent calls, Z-key cancel, buffer reset in finally block
Demo preparation	Pre-loaded audio for video shoot, non-blocking parallel playback

7.1 Notable Engineering Decisions

Per-press calibration

The earliest firmware calibrated on startup by having the user hold the pen still for two seconds. This was awkward, forgotten, and broke when the user changed grip between strokes. The insight was that calibration should happen at the moment of action, not at startup. Now the system snapshots the IMU state on the first frame of every button press. The user never thinks about calibration.

Image-based stroke capture

An alternative approach would record the trajectory of the cursor as a time series and classify that sequence directly. The image-based approach was chosen because it leverages the entire existing ecosystem of image classification models and tooling, requires no custom sequence model architecture, and is robust to variation in drawing speed.

Phrase shortcuts with unordered matching

For demonstration phrases, calling a language model on every input adds latency and API dependency. The shortcut dictionary uses Python frozenset keys, which means the words {we, not, communicate, mouth} match regardless of the order the user draws them. This is the right default for air-writing because there is no enforced word order — the user draws what comes naturally.

8. Roadmap: Version 2 and Beyond

The real vision

Version 1 proves the pipeline. Version 2 breaks the word-by-word constraint entirely.

8.1 Direct Teeline Translation (V2 Core)

The current classifier is trained to recognise discrete word gestures drawn one at a time. This works, but it is architecturally limited: the vocabulary is fixed at training time, adding new words requires new data collection and retraining, and the user must draw complete, isolated gestures.

The planned Version 2 model works differently. Instead of classifying individual strokes, it will interpret continuous Teeline shorthand writing — the same way a journalist reads shorthand from a notepad. The model will receive a stream of motion data and produce a stream of words, without requiring the user to separate each word with a button press. This is a fundamentally more natural interaction and an unbounded vocabulary.

This approach was identified and designed during the hackathon. It was not built during the event because it requires a purpose-built dataset of continuous Teeline air-writing and a sequence-to-sequence model architecture — work that could not be completed in the available time. It is the primary technical goal of the next development phase.

8.2 Hardware Miniaturisation

The current prototype uses a development board in a wand form factor. The next hardware iteration will move to a compact microcontroller in a purpose-built casing closer to the size of an actual pen or a finger ring. The firmware is already sufficiently lean to run on a smaller chip.

8.3 Wireless Operation

A WiFi UDP communication layer was prototyped during development and is architecturally complete. The remaining work is latency optimisation on the ESP32 power-save settings. Once stable, the cable can be removed entirely.

8.4 Expanded Use Cases

The gesture layer is not limited to assistive communication. The same wand, with the same or extended gesture vocabulary, can serve as an input device for any system an AI agent can control: smart home commands, device control, silent communication in environments where voice is not appropriate. The interaction model — one hand, a small set of learnable gestures, real-time AI interpretation — is general-purpose.

9. Project File Inventory

File	Purpose
AIRTALK.ino	Current Arduino firmware — per-press calibration, single snprintf output
draw.py	Core air-writing visualiser — serial input, pygame canvas, motion processing
capture_record.py	Training data collection — session labelling, auto-indexed PNG saves
infer.py	Real-time inference — CNN classification, Groq LLM + TTS
gemini-infer.py	Inference variant using Google Gemini Direct API
openrtr-gemini-infer.py	Inference variant using OpenRouter API (current primary)
for_vid .py	Video demo variant — pre-loaded audio, non-blocking playback
train.ipynb	Full ML training notebook — MobileNetV2, 80/20 split, confusion matrix
model_best.pt	Trained MobileNetV2 weights (~9 MB)
captures/	Training dataset — 10 word classes, ~421 labelled images
requirements.txt	Python dependencies
.env	API keys (not committed to version control)

10. Dependencies and Setup

10.1 Python dependencies

- pygame >= 2.5.0 — real-time visualisation and audio playback
- pyserial >= 3.5 — USB serial communication (install as pyserial, not serial)
- torch >= 2.0.0 — CNN inference
- torchvision >= 0.15.0 — MobileNetV2 architecture and transforms
- pillow >= 10.0.0 — image preprocessing
- numpy >= 1.24.0 — array operations
- requests — OpenRouter HTTP calls
- google-generativeai >= 0.8.0 — Gemini Direct API variant

10.2 Quick start

Visualise (no inference): `python3 draw.py`

Collect training data: `python3 capture_record.py`

Run inference: `python3 openrtr-gemini-infer.py`

Train model: `jupyter notebook train.ipynb`

Curiosity before certainty.

Affan Sajid | airtalk | Google Physical AI Hackathon 2026